source_file

sourceItem sourceItem sourceItem sourceItem

funcDef

method funcSignature body

id: x ( ) var varDecl block

idList : typeRef ;

id: a , id: b int

begin expr_stmt end ;

postfix ;

id: send_byte (literal)

char: 'x'

funcDef

method funcSignature body

id: y ( ) block

begin expr_stmt end ;

postfix ;

id: send_byte (literal)

char: 'y'

funcDef

method funcSignature body

id: z ( ) block

begin expr_stmt end ;

postfix ;

id: send_byte (literal)

char: 'z'

funcDef

method funcSignature body

id: main ( ) var varDecl varDecl varDecl block

idList : typeRef ;

id: argc int

idList : typeRef ;

id: argv array [ ] of typeRef

string

idList : typeRef ;

id: i , id: a , id: b int

begin expr_stmt while_statement expr_stmt expr_stmt if_statement while_statement if_statement if_statement while_statement expr_stmt expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: '1'

while literal do block

num: 1 begin expr_stmt while_statement expr_stmt if_statement expr_stmt do_statement expr_stmt expr_stmt if_statement expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: '2'

while literal do block

num: 2 begin expr_stmt if_statement end ;

postfix ;

id: send_byte (literal)

char: '3'

if literal then expr_stmt else if_statement

num: 3 postfix ;

id: send_byte (literal)

char: 'b'

if literal then expr_stmt else block

num: 4 postfix ;

id: send_byte (literal)

char: 'c'

begin expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: 'l'

postfix ;

id: send_byte (literal)

char: '4'

postfix ;

id: send_byte (literal)

char: '5'

if literal then expr_stmt else if_statement

num: 5 postfix ;

id: send_byte (literal)

char: 'b'

if literal then expr_stmt else expr_stmt

num: 6 postfix ;

id: send_byte (literal)

char: 'c'

postfix ;

id: send_byte (literal)

char: '6'

postfix ;

id: send_byte (literal)

char: '7'

repeat block while literal ;

begin expr_stmt if_statement end ;

postfix ;

id: send_byte (literal)

char: '8'

if literal then expr_stmt

num: 9 postfix ;

id: send_byte (literal)

char: 'b'

num: 10

postfix ;

id: send_byte (literal)

char: '1'

postfix ;

id: send_byte (literal)

char: '1'

if literal then expr_stmt else if_statement

num: 11 postfix ;

id: send_byte (literal)

char: 'b'

if literal then expr_stmt else block

num: 12 postfix ;

id: send_byte (literal)

char: 'c'

begin expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: '1'

postfix ;

id: send_byte (literal)

char: '2'

postfix ;

id: send_byte (literal)

char: '1'

postfix ;

id: send_byte (literal)

char: '3'

postfix ;

id: send_byte (literal)

char: '1'

postfix ;

id: send_byte (literal)

char: '4'

if relational then block else block

id: a > literal

num: 3

begin expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: '1'

postfix ;

id: send_byte (literal)

char: 'n'

begin expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: '2'

postfix ;

id: send_byte (literal)

char: 'n'

while relational do block

id: i < literal

num: 10

begin if_statement end ;

if relational then assignment else if_statement

id: a > id: b id: i := add ;

id: i + literal

num: 2

if mul then expr_stmt else expr_stmt

literal * literal

num: 2 num: 3

postfix ;

id: send_byte (literal)

char: 'b'

postfix ;

id: send_byte (literal)

char: 'c'

if literal then if_statement

num: 1 if literal then if_statement else expr_stmt

num: 2 if literal then expr_stmt else expr_stmt

num: 3 id: x ; id: y ;

id: z ;

if relational then block else block

id: a > literal

num: 3

begin expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: '1'

postfix ;

id: send_byte (literal)

char: 'n'

begin expr_stmt expr_stmt expr_stmt expr_stmt expr_stmt expr_stmt expr_stmt expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: '2'

postfix ;

id: send_byte (literal)

char: 'n'

postfix ;

id: send_byte (literal)

char: '2'

postfix ;

id: send_byte (literal)

char: '2'

postfix ;

id: send_byte (literal)

char: 'n'

postfix ;

id: send_byte (literal)

char: '2'

postfix ;

id: send_byte (literal)

char: '2'

postfix ;

id: send_byte (literal)

char: '2'

postfix ;

id: send_byte (literal)

char: 'n'

while relational do block

id: a < literal

num: 3

begin expr_stmt expr_stmt end ;

postfix ;

id: send_byte (literal)

char: '3'

postfix ;

id: send_byte (literal)

char: 'n'

postfix ;

id: send_byte (literal)

char: 'H'

postfix ;

id: send_byte (literal)

char: 'i'

postfix ;

id: send_byte (literal)

char: 'n'